

Obecné OOP a Třídy

1. **Třída vs. objekt:** Třída je abstraktní předpis (šablona), zatímco objekt je její konkrétní instance v paměti.
2. **Viditelnost prvků:** Určuje se modifikátory přístupu public (veřejné), private (soukromé) a protected (chráněné).
3. **Instanční vs statické prvky:** Instanční (objektové) prvky patří každé instanci zvlášť, statické (třídní) patří celé třídě a všechny objekty je sdílí.
4. **Protected vs. private:** K private prvkům má přístup jen samotná třída, k protected mají navíc přístup i její potomci.
5. **Polymorfismus:** Schopnost volat stejnou metodu na objektech různých typů, přičemž každý zareaguje svou vlastní implementací (využívá virtuální metody).
6. **Bázová třída:** Rodičovská třída v hierarchii dědičnosti, která poskytuje společné vlastnosti a rozhraní svým potomkům.
7. **Plně virtuální metoda:** Metoda, kterou předek nedefinuje a potomek ji MUSÍ implementovat.

```
C++  
  
virtual void metoda() = 0;
```

8. **Implicitní konstruktor:** Bezparametrický konstruktor, který za tebe automaticky vygeneruje kompilátor, pokud si nenapišeš žádný vlastní.
9. **Kopírovací konstruktor:** Vytváří nový objekt jako přesnou kopii jiného, již existujícího objektu.

```
C++  
  
Trida(const Trida& jina);
```

10. **Možnosti dědění:** Dědit lze public (nejčastější), protected nebo private.

```
C++  
  
class Potomek : public Rodic {};
```

11. **Virtuální dědění:** Zabraňuje tomu, aby při vícenásobném dědění vzniklo v potomkovi více instancí stejného společného předka (řeší diamantový problém).

12. **Co potomek nedědí:** Konstruktory, destruktory, přetížené operátory přiřazení a explicitní deklarace friend.

13. **Přetížení (Overload):** Funkce se stejným názvem, ale s různým počtem nebo typem parametrů.

```
C++  
  
void tisk(int a);  
void tisk(double a);
```

14. **Přepsání (Override) vs Přetížení:** Přetížení se liší parametry ve stejné třídě, přepsání znamená úpravu chování existující virtuální metody rodiče (má stejné parametry).

15. **Kompozice:** Silná vazba (objekt vlastní jiný objekt); zánikem celku zaniká i část.

```
C++  
  
class Auto { Motor m; };
```

16. **Asociace:** Volná vazba (objekt jen "zná" jiný objekt); objekty existují nezávisle na sobě.

```
C++  
  
class Clovek { Auto* mojeAuto; };
```

17. **Klíčové slovo this:** Je to skrytý ukazatel na aktuální objekt, nad kterým byla právě zavolána metoda.

18. **Reference vs ukazatel:** Reference je neměnný alias (jiné jméno) existující proměnné, zatímco ukazatel je proměnná nesoucí paměťovou adresu, lze ji přesměrovat a může být prázdná (nullptr).

19. **Mělká vs hluboká kopie:** Mělká kopíruje jen paměťové adresy (vzniká sdílení), hluboká alokuje novou paměť a reálně zkopíruje i původní hodnoty.

⚙️ Programování v C++

20. Načtení int a ošetření:

```
C++  
  
int x;  
if (std::cin >> x) { /* Vstup v pořádku */ } else { /* Chyba - zadáno ne-číslo */ }
```

C++

```
int x;
```

```
if (std::cin >> x) { /* Vstup v pořádku */ } else { /* Chyba - zadáno ne-číslo */ }
```

21. **Přetížení operátoru:** Definuje, co se stane, když se standardní operátor použije na tvůj vlastní typ.

```
C++  
  
Bod operator+(const Bod& a, const Bod& b) { return Bod(a.x + b.x, a.y + b.y); }
```

22. **Co nelze přetížit:** Operátory . (tečka), *, :: (čtyřtečka), ?: (ternární) a typové zjišťování (sizeof, typeid).

23. **Konstruktor a destruktork:** Konstruktor se volá automaticky při vytvoření (alokaci) objektu, destruktork při jeho zániku (opuštění bloku nebo delete).

24. **Zásobník (Stack) vs halda (Heap):** Zásobník je rychlý, lokální a uvolňuje se sám; halda je pro dynamická data, jejichž paměť si programátor spravuje ručně (new/delete).

25. **delete vs delete[]:** delete maže jen jeden objekt, delete[] maže celé pole dynamicky alokovaných objektů.

26. **Návratová hodnota destruktorku:** Nemá vůbec žádnou (nedeklaruje se u něj ani void).

27. **Zajištění při dynamické alokaci:** Musíš vždy zajistit následné uvolnění paměti, jinak vznikne paměťový únik (memory leak).

```
C++  
  
int* p = new int;  
delete p; // tohle nesmí chybět
```

28. **Výjimka:** Mechanismus pro elegantní zachycení a zpracování neočekávaných chyb mimo běžný kód.

```
C++  
  
try { throw std::runtime_error("Chyba!"); }  
catch (const std::exception& e) { /* řešení chyby */ }
```

29. **Kolik výjimek lze ošetřit:** Teoreticky neomezené množství kaskádováním bloků catch.

30. **Generická funkce (šablona):** Funkce, která dokáže pracovat s univerzálním zástupným datovým typem (bez nutnosti psát ji pro int a double zvlášť).

31. **Explicitní instance:** Slouží k vynucení překladu šablony pro jeden konkrétní typ už při kompilaci (hlavně kvůli zrychlení a oddělení implementace od hlaviček).

32. **Parametrizace šablon:** Šabloně obvykle podstrkáváš buď datové typy (typename T), nebo konstantní hodnoty (např. velikost pole int N).

33. **Problém vícenásobného dědění:** Pokud třída dědí ze dvou, které mají stejného předka, vznikne nejednoznačnost; zabrání se tomu "virtuálním děděním" (virtual public Predka).

34. **Zapouzdření:** Skrytí interních dat objektu před světem a definování striktního rozhraní pro manipulaci s nimi (ochrana před narušením stavu).

35. **Implicitní hodnota argumentu:** Umožňuje definovat přednastavenou hodnotu, takže při volání ji nemusíš zadávat.

```
C++  
  
void pozdrav(int pocet = 1); // lze volat jako pozdrav();
```

36. **class vs struct:** Ve struct jsou veškeré proměnné a metody defaultně public, v class jsou defaultně private.

37. **Konverzní konstruktor:** Konstruktor beroucí jen 1 parametr, který může kompilátor tajně využít k převodu datového typu (zakazuje se klíčovým slovem explicit).

38. **Statická vs dynamická vazba:** Statická se vyhodnocuje už při kompilaci, dynamická se rozhoduje až za běhu programu na základě přesného typu objektu pod ukazatelem.

39. **Klíčové slovo auto:** Překladač díky němu sám odhadne datový typ; lze použít jen tehdy, pokud je proměnná okamžitě inicializována (rovnou se jí něco přiřadí).

40. **Přesouvací (move) konstruktor:** Místo drahého kopírování "ukradne" dynamická data z dočasného (zanikajícího) objektu přehozením ukazatelů.
41. **Pravidlo tří:** Jakmile potřebuješ vlastní destruktorku, kopírovací konstruktor NEBO kopírovací operátor `operator=`, budeš pravděpodobně potřebovat implementovat všechny tři.
42. **Lambda funkce:** Bezejmenná lokální funkce vytvořená přímo na místě (často se předává jako parametr jiné funkci).

```
C++  
  
auto secti = [](int a, int b) { return a + b; }; // Sečte dvě čísla
```

STL kontejnery

43. **Kontejner:** Datová struktura ze standardní knihovny pro bezpečné uchovávání kolekcí hodnot/objektů.
44. **Sekvenční vs asociativní:** Sekvenční (např. `vector`) udržují přesné pořadí podle toho, jak data vkládáš; asociativní (např. `map`) data ihned vyhledávají a organizují podle klíče.
45. **Vlastnosti `std::vector`:** Souvislá paměť, extrémně rychlý přístup na index, dynamicky se zvětšuje, ale pomalé vkládání doprostřed.
46. **Vlastnosti `std::map`:** Páry klíč-hodnota ve stromové struktuře, klíče se automaticky řadí a nesmí se opakovat.
47. **Vlastnosti `std::list`:** Zřetěžený seznam, velmi rychlé vkládání a mazání kdekoli, ale pomalé průchody (nelze se na prvek dotázat přímo přes index).
48. **Typ klíče `std::map`:** Prakticky jakýkoliv, u kterého lze jasně určit řazení (musí pro něj fungovat operátor `<`).
49. **Vlastní struktura jako klíč mapy:** Musíš pro svou vlastní třídu přetížit operátor `<` nebo mapě předat hodnotící funkci (funktor), aby věděla, jak klíče seřadit.
50. **Význam `std::pair`:** Jednoduchý pomocný objekt, který spojuje jakékoliv dvě různé hodnoty do jednoho balíčku (klíčový prvek uvnitř mapy).

51. **Iterátor:** Objekt (chytrý ukazatel) sloužící k procházení prvků uvnitř STL kontejneru nezávisle na jeho konkrétní vnitřní stavbě.

C++

```
std::vector<int>::iterator it = v.begin();
```

52. **Konstantní vs nekonstantní iterátor:** Přes nekonstantní iterátor můžeš hodnoty v kontejneru číst i upravovat, přes const_iterator pouze číst.

53. **Range-for cyklus:** Zjednodušený for cyklus, který projde celý kontejner automaticky od začátku do konce.

C++

```
for (int cislo : mujVektor) { std::cout << cislo; }
```

54. **Algoritmus sort:**

C++

```
std::vector<int> v = {9, 2, 5};  
std::sort(v.begin(), v.end()); // v nyní obsahuje {2, 5, 9}
```